

Flink Models 模块

核心流程与设计说明

基于 Apache Flink 2.3-SNAPSHOT 源码分析
flink-models 模块完整解读 (OpenAI + Triton)

模块路径: `flink-models/{flink-model-openai, flink-model-triton}`
共 18 个核心 Java 源文件 | 25 个 Java 文件(含测试)

目录

- 一、模块概览与架构设计
- 二、核心类设计说明
- 三、OpenAI Chat Completion 调用流程
- 四、Triton Inference 推理请求流程
- 五、ML_PREDICT 异步预测时序图
- 六、错误处理策略
- 七、上下文溢出处理 (OpenAI)
- 八、Flink-Triton 类型映射
- 九、引用计数客户端池
- 十、参数与配置体系
- 十一、SPI 机制与扩展点
- 十二、关键设计模式

一、模块概览与架构设计

1.1 模块定位

flink-models 是 Apache Flink 2.3 版本新增的模型推理服务集成模块，旨在通过 Flink SQL 的 ML_PREDICT 函数将外部 AI/ML 模型推理能力无缝嵌入流处理管道。该模块采用 Maven 多模块结构，包含两个子模块: flink-model-openai (集成 OpenAI API，支持 Chat Completion 和 Embedding) 和 flink-model-triton (集成 NVIDIA Triton Inference Server，支持通用模型推理)。两个子模块均基于 Flink 的 AsyncPredictFunction 异步预测框架，通过 SPI 机制自动注册为 Flink Table Factory。

1.2 模块包结构

子模块	包名	核心类数	职责
flink-model-openai	o.a.f.model.openai	7	OpenAI API 集成 (Chat + Embedding)
flink-model-triton	o.a.f.model.triton	7	Triton Inference Server 集成
flink-model-triton	o.a.f.model.triton.exception	5	Triton 异常层次体系

1.3 依赖关系

子模块	关键依赖	版本	用途
openai	com.openai:openai-java	1.6.1	OpenAI 官方 Java SDK (异步)
openai	com.knuddels:jtokkit	1.1.0	Token 编码计数
triton	com.squareup.okhttp3:okhttp	4.12.0	HTTP 客户端
triton	jackson-core/databind	2.15.2	JSON 序列化/反序列化
共用	flink-core/table-api-java	2.3-SNAPSHOT	Flink 核心 (provided)

1.4 核心类层次结构

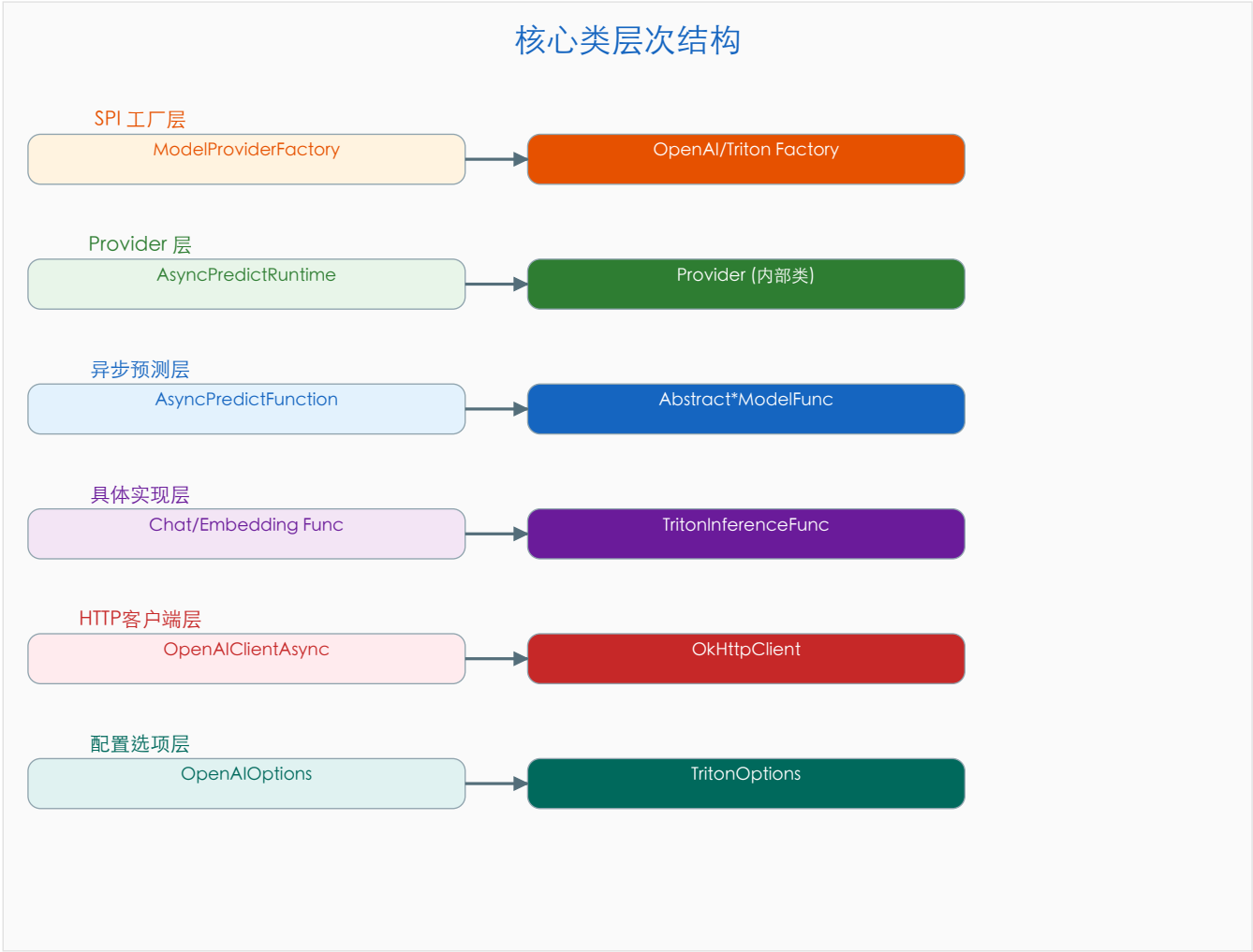


图 1-1: 核心类层次结构

1.5 整体架构概览



二、核心类设计说明

2.1 OpenAIModelProviderFactory (SPI 入口, 约107行)

实现 `ModelProviderFactory` 接口, SPI 标识符为 `"openai"`。根据 `endpoint` URL 后缀自动路由到不同的 `ModelFunction` 实现。

- `factoryIdentifier()`: 返回 `"openai"`, 用于 Flink Table Factory SPI 自动发现
- 路由逻辑: `endpoint` 以 `"chat/completions"` 结尾 -> `OpenAIChatModelFunction`; 以 `"embeddings"` 结尾 -> `OpenAIEmbeddingModelFunction`
- 创建 Provider 内部类: 实现 `AsyncPredictRuntimeProvider`, 持有具体的 `AsyncPredictFunction` 实例
- 必选参数: `endpoint`, `api-key`, `model`; 可选: `max-context-size`, `error-handling-strategy`, `system-prompt`, `temperature` 等

2.2 AbstractOpenAIModelFunction (抽象基类, 约338行)

继承 `AsyncPredictFunction`, 管理 HTTP 客户端生命周期、Token 上下文控制和错误处理。

- `open()`: 通过 `OpenAIUtils` 获取引用计数的 `OpenAIClientAsync` + 初始化 `jtokkit Tokenizer`
- `close()`: 释放客户端引用 (引用计数减一)
- `asyncPredict()`: 模板方法 — `null` 检查 -> `token` 限制 -> 调用子类 `asyncPredictInternal()`
- `handleErrorsAndRespond()`: `FAILOVER`(抛异常) / `IGNORE`(返空+元数据)
- 内部枚举: `ErrorHandlingStrategy`(`RETRY`/`FAILOVER`/`IGNORE`), `RetryFallbackStrategy`(`FAILOVER`/`IGNORE`)
- `ErrorMessageMetadata`: `error-string`(`STRING`) / `http-status-code`(`INT`) / `http-headers-map`(`MAP`)

2.3 OpenAIChatModelFunction (Chat 完成, 约160行)

继承 `AbstractOpenAIModelFunction`, 实现 `OpenAI Chat Completion API` 调用。输入: 单列 `STRING`; 输出: 单列 `STRING`。

- 构建 `ChatCompletionCreateParams`: `system prompt` + `user message` + `model`
- 可选参数: `temperature`, `topP`, `stop`, `maxTokens`, `presencePenalty`, `n`, `seed`, `responseFormat`(`TEXT`/`JSON_OBJECT`)
- 响应处理: 提取每个 `choice` 的 `message.content` 生成 `GenericRowData`

2.4 OpenAIEmbeddingModelFunction (向量嵌入, 约116行)

输入: 单列 `STRING`; 输出: 单列 `ARRAY<FLOAT>` (嵌入向量)。构建 `EmbeddingCreateParams`, 将 `embedding` 向量转为 `GenericArrayData`。

2.5 TritonModelProviderFactory (SPI 入口, 约95行)

SPI 标识符 "triton"。必选: endpoint, model-name; 可选: model-version, timeout, flatten-batch-dim, priority, sequence-id, compression, auth-token, custom-headers。

2.6 TritonInferenceModelFunction (核心推理, 约455行)

继承 AbstractTritonModelFunction，实现 Triton v2 REST API 推理。这是 Triton 子模块最核心的类。

- asyncPredict(): 构建请求 -> HTTP POST -> 解析响应 -> 返回 RowData
- buildInferenceRequest(): 序列化 -> 计算shape -> flattenBatchDim -> sequence/priority
- URL: {endpoint}/v2/models/{name}/versions/{version}/infer
- gzip 压缩: 可复用 ByteArrayOutputStream 缓冲区
- handleErrorResponse(): 400->SchemaException, 404->检查模型名, 401/403->检查认证, 5xx->ServerException

2.7 AbstractTritonModelFunction (抽象基类, 约323行)

- open()/close(): 管理 OkHttpClient 引用计数
- validateSingleColumnSchema(): 确保单列 + Triton 类型兼容
- 禁止嵌套数组 (ARRAY<ARRAY<T>>)，提供类型转换建议

2.8 其他核心类

类名	职责
ContextOverflowAction	上下文溢出枚举(6种): TRUNCATED_TAIL/HEAD + SKIPPED (各有+LOG变体)
OpenAIUtils	引用计数池: Key=(baseUrl,apiKey), 复用 OpenAIClientAsync
TritonDataType	Triton 数据类型枚举(13种): BOOL, INT8-64, FP16-64, BYTES
TritonTypeMapper	Flink<->Triton 类型映射 + 序列化/反序列化 + Shape计算
TritonUtils	引用计数池: Key=timeoutMs + buildInferenceUrl()
TritonException	异常基类: isRetryable() + getCategory() + ErrorCategory
TritonClientException	4xx, isRetryable=false
TritonServerException	5xx, 503/504 可重试
TritonNetworkException	IOException, isRetryable=true
TritonSchemaException	Shape不匹配, expected vs actual

三、OpenAI Chat Completion 调用流程

OpenAI Chat Completion 是 flink-model-openai 模块最核心的功能。通过 OpenAIChatModelFunction 将用户文本发送到 OpenAI API，获取 AI 生成的回复文本。



图 3-1: OpenAI Chat Completion 调用流程

关键实现细节:

- asyncPredict() 首先检查输入是否为 null，若为 null 直接返回 null

- Token 计数使用 jtokkit 的 Encoding 对象编码为 token ID 数组后计数
- 超过 maxContextSize 时按 ContextOverflowAction 策略截断/跳过 (默认 TRUNCATED_TAIL)
- 构建 ChatCompletionCreateParams: system prompt (可选) + user message + model
- responseFormat 支持 TEXT 和 JSON_OBJECT 两种模式
- 异步调用返回 CompletableFuture，提取每个 choice 的 message.content

四、Triton Inference 推理请求流程

通过 TritonInferenceModelFunction 将 Flink 数据发送到 NVIDIA Triton Inference Server 执行模型推理。采用 Triton v2 REST API 协议。

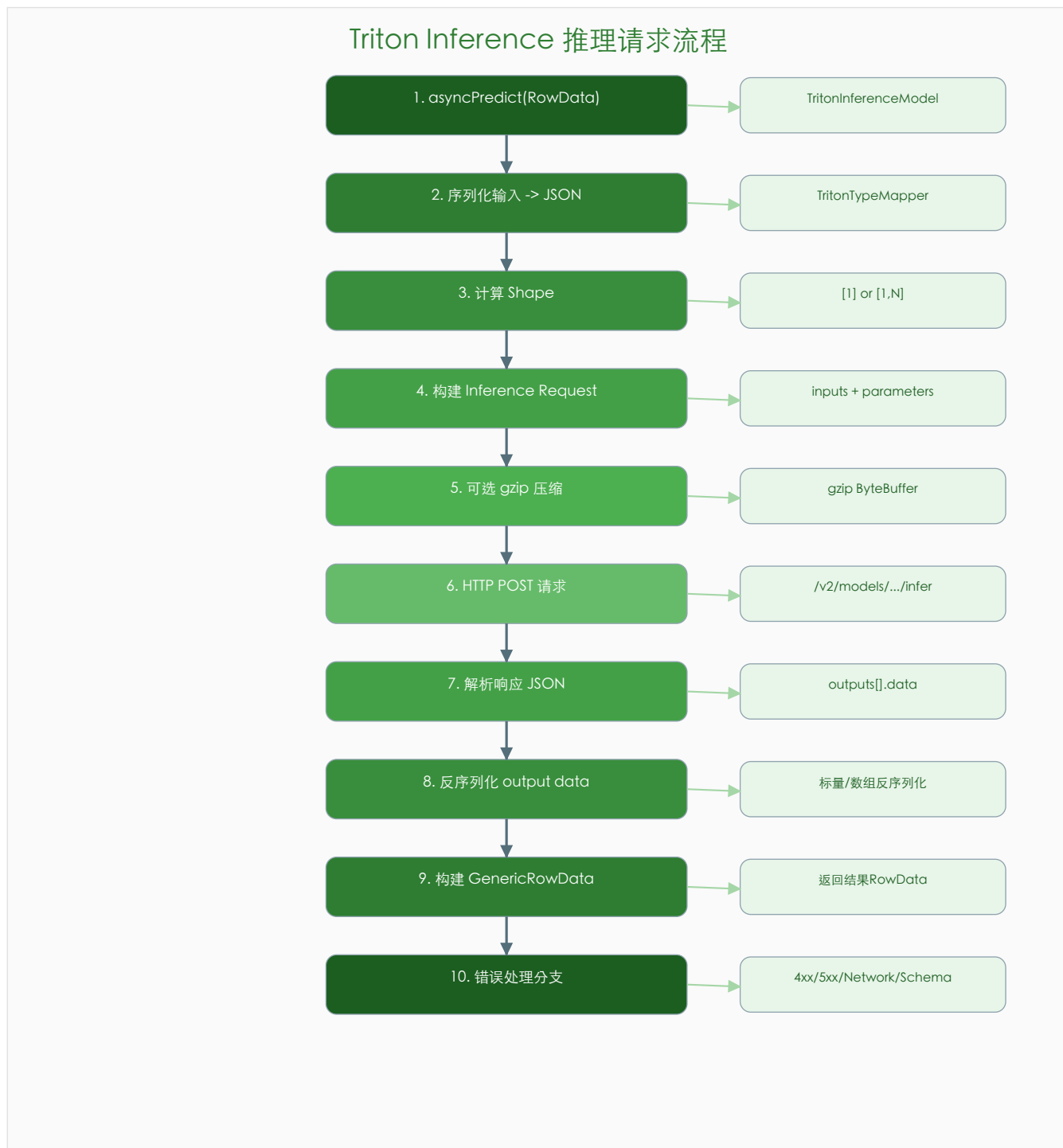


图 4-1: Triton Inference 推理请求流程

关键实现细节:

- `TritonTypeMapper.serializeToJsonArray()` 序列化为 JSON 数组 (数组类型展开)

- Shape: 标量->[1], 数组->[1,N]; flattenBatchDim=true 时 [1,N]->[N]
- 请求 JSON: {"inputs":[{"name":...,"datatype":...,"shape":...,"data":...}], "parameters":{..}}
- gzip 压缩使用可复用缓冲区; 支持 Bearer Token + 自定义 Headers
- 响应: outputs[0].data 标量取首元素/数组完整反序列化
- sequence-id/start/end 参数支持有状态模型推理

五、ML_PREDICT 异步预测时序图

Flink SQL 的 ML_PREDICT 函数触发异步预测。基于 CompletableFuture 实现非阻塞异步执行。



图 5-1: ML_PREDICT 异步预测时序图

- open(): 通过引用计数池获取 HTTP 客户端 + 初始化 Tokenizer/TypeMapper
- asyncPredict(): RowData -> 预处理 -> 构建请求 -> 异步 HTTP -> 解析响应
- 错误处理: OpenAI 按 ErrorHandlerStrategy; Triton 抛出分类异常
- close(): 释放客户端引用，归零时关闭连接

六、错误处理策略

OpenAI 提供可配置三级策略+元数据回传; Triton 实现完整异常层次+可重试判断。

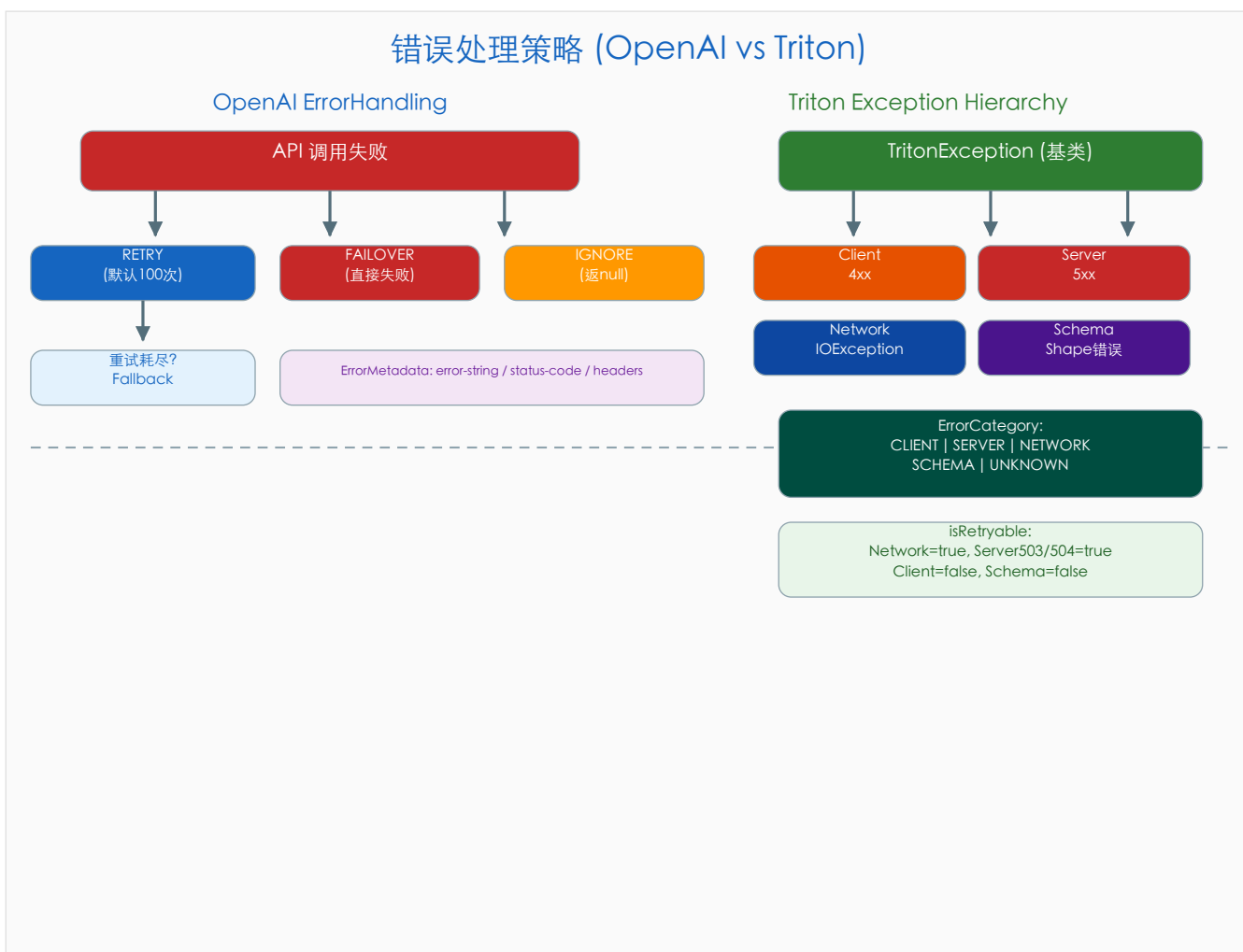


图 6-1: 错误处理策略对比

6.1 OpenAI 错误处理

- RETRY(默认): 自动重试 retry-num 次(默认100)
- FAILOVER: 直接抛 RuntimeException 触发 Flink Failover
- IGNORE: 忽略错误返回 null + ErrorMessageMetadata 元数据
- RetryFallbackStrategy: 重试耗尽后 FAILOVER(默认)/IGNORE

6.2 Triton 异常层次

异常类	触发条件	isRetryable	ErrorCategory
TritonClientException	4xx	false	CLIENT_ERROR
TritonServerException	5xx	503/504:true	SERVER_ERROR
TritonNetworkException	IOException	true	NETWORK_ERROR
TritonSchemaException	Shape/Type不匹配	false	SCHEMA_ERROR

- 400 Bad Request: 含 shape 关键词 -> TritonSchemaException
- 404: 提示检查 model-name/version; 401/403: 提示检查 auth-token
- 503/504: isRetryable=true; 其他 5xx: isRetryable=false

七、上下文溢出处理 (OpenAI)

通过 jtokkit 库进行 Token 编码和计数，超过 max-context-size 时按策略截断/跳过。

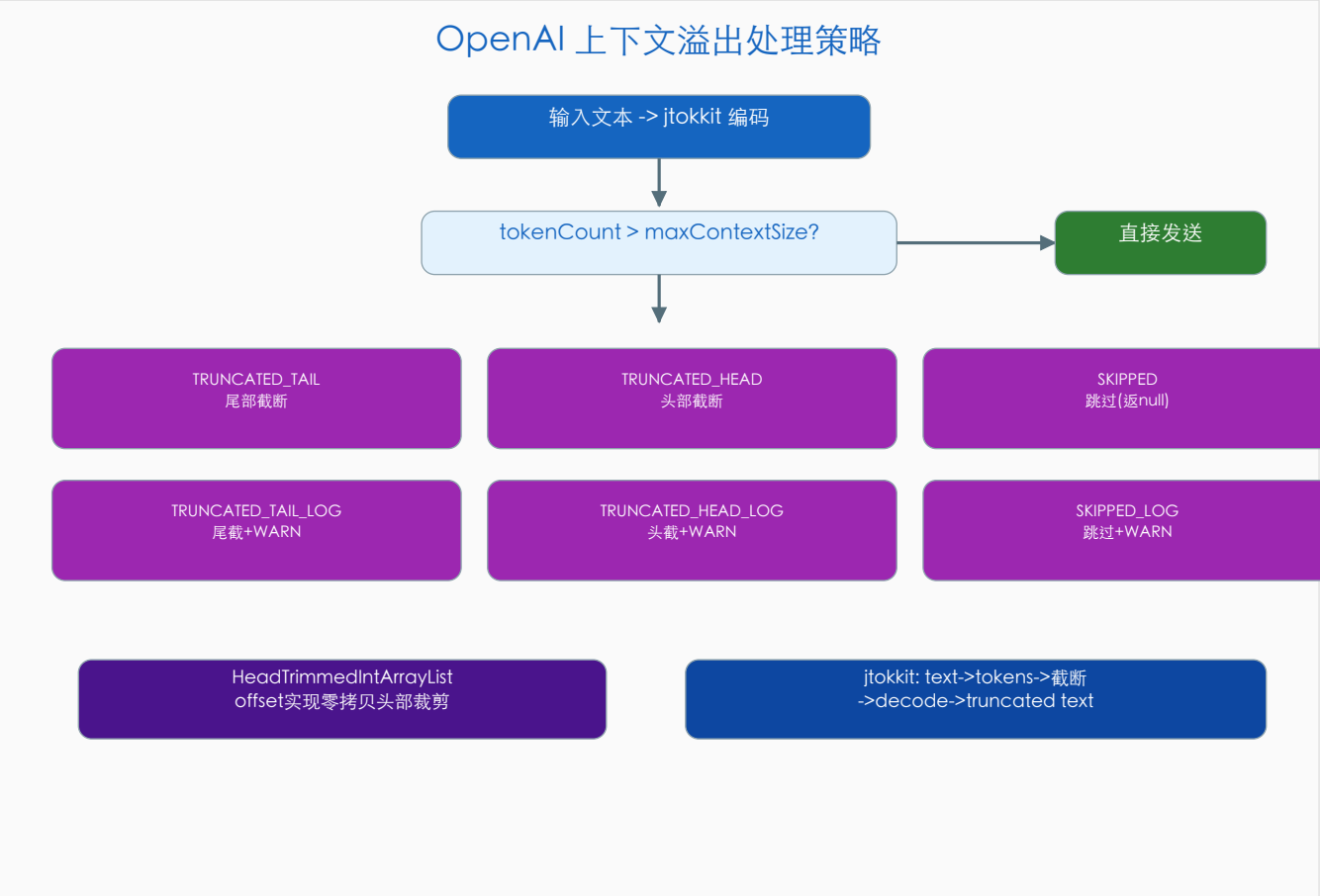


图 7-1: 上下文溢出处理策略

枚举值	行为	日志
TRUNCATED_TAIL	从尾部截断: 保留前 N 个 token	无
TRUNCATED_TAIL_LOG	同上	WARN
TRUNCATED_HEAD	从头部截断: 保留后 N 个 token	无
TRUNCATED_HEAD_LOG	同上	WARN
SKIPPED	跳过该行返回 null	无
SKIPPED_LOG	同上	WARN

HeadTrimmedIntArrayList 优化: 继承 IntArrayList，通过 offset 实现零拷贝头部裁剪。size()=originalSize-offset, get(i)=array[i+offset]。

八、Flink-Triton 类型映射

TritonTypeMapper 实现 Flink LogicalType 与 Triton DataType 的双向映射、序列化和反序列化。

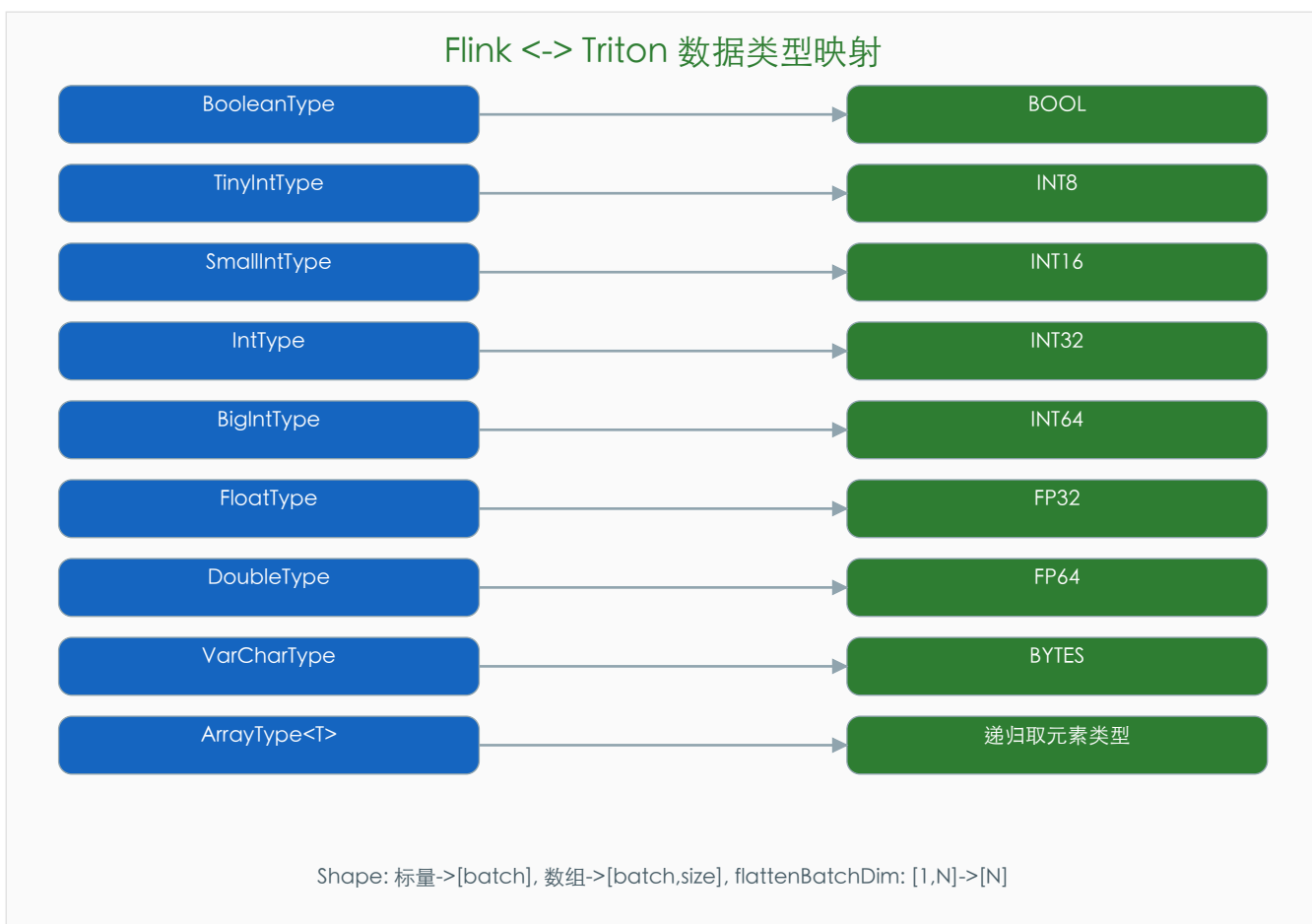


图 8-1: Flink-Triton 数据类型映射

8.1 序列化 (Flink -> Triton)

- 标量: 直接序列化为 [value]; 数组: 展平为 [v1,v2,...,vN]
- VarChar/String 编码为 UTF-8 对应 Triton BYTES 类型
- 使用原始类型数组 (int[],float[]) 避免装箱开销

8.2 反序列化 (Triton -> Flink)

- 标量: 从 data 取首元素; 数组: 完整反序列化为 ArrayData
- 整数用 Number.intValue()/longValue(); 浮点用 floatValue()/doubleValue()

8.3 Shape 计算

calculateShape(): 标量->[batchSize], 数组->[batchSize,arraySize]。flattenBatchDim=true 且 batch=1 时 [1,N]->[N]。

九、引用计数客户端池

OpenAIUtils 和 TritonUtils 实现引用计数 HTTP 客户端池，高并行度下多 Subtask 共享同一客户端。

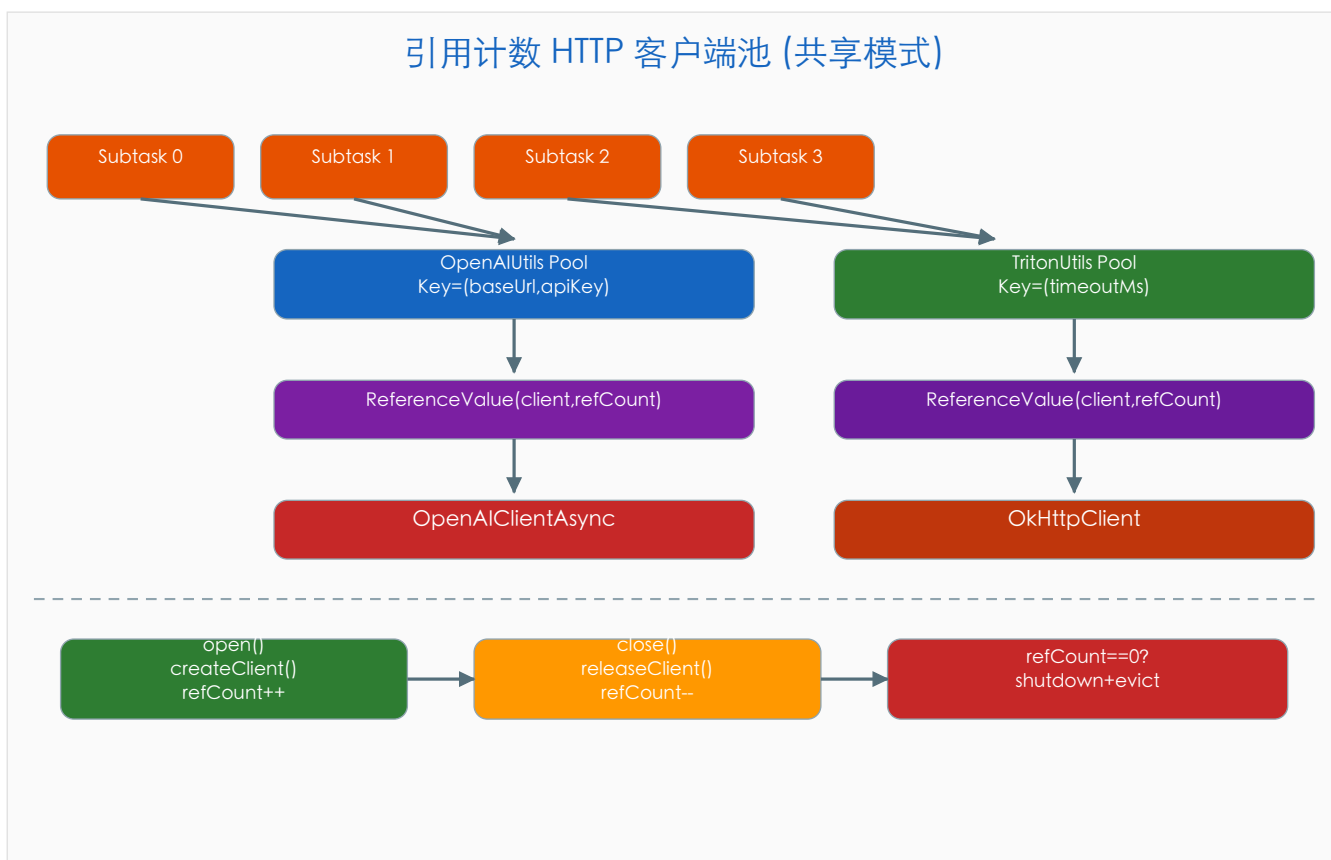


图 9-1: 引用计数 HTTP 客户端池

9.1 OpenAIUtils

- Key=(baseUrl,apiKey); createAsyncClient(): 命中refCount++ / 未命中创建新客户端
- releaseAsyncClient(): refCount-- 归零关闭; synchronized 线程安全

9.2 TritonUtils

- Key=timeoutMs; connect/read/write 超时统一; 开启连接失败重试
- releaseHttpClient(): 归零时 shutdown dispatcher + evict 连接池
- buildInferenceUrl(): 智能 URL 规范化，自动补全 /v2/models/.../infer

十、参数与配置体系

10.1 OpenAI 配置项 (OpenAIOptions)

分类	配置项	类型	默认值	说明
通用	endpoint	String	必选	OpenAI API URL
通用	api-key	String	必选	API 密钥
通用	model	String	必选	模型名(gpt-3.5等)
上下文	max-context-size	Integer	无	最大 token 数
上下文	context-overflow-action	Enum	TRUNCATED_TAIL	溢出策略
错误	error-handling-strategy	Enum	RETRY	错误策略
错误	retry-num	Integer	100	重试次数
错误	retry-fallback-strategy	Enum	FAILOVER	重试耗尽回退
Chat	system-prompt	String	无	系统提示词
Chat	temperature	Double	无	温度(0-2)
Chat	top-p	Double	无	核采样
Chat	max-tokens	Integer	无	最大生成token
Chat	n	Integer	无	候选数
Chat	response-format	Enum	无	TEXT/JSON_OBJECT
Embed	dimension	Integer	无	向量维度

10.2 Triton 配置项 (TritonOptions)

分类	配置项	类型	默认值	说明
基础	endpoint	String	必选	Triton Server URL
基础	model-name	String	必选	模型名称
基础	model-version	String	latest	模型版本
基础	timeout	Duration	30s	HTTP超时
高级	flatten-batch-dim	Boolean	false	[1,N]->[N]
高级	priority	Integer	无	优先级(0-255)
序列	sequence-id	String	无	序列ID
序列	sequence-start	Boolean	false	序列开始
序列	sequence-end	Boolean	false	序列结束
HTTP	compression	String	无	gzip
HTTP	auth-token	String	无	Bearer Token
HTTP	custom-headers	Map	无	自定义头

十一、SPI 机制与扩展点

11.1 SPI 注册

两个子模块通过 META-INF/services/org.apache.flink.table.factories.Factory 文件注册:

- flink-model-openai: org.apache.flink.model.openai.OpenAIModelProviderFactory
- flink-model-triton: org.apache.flink.model.triton.TritonModelProviderFactory

11.2 Shade 机制

两个模块均使用 maven-shade-plugin 对第三方依赖进行 relocation，避免与用户依赖冲突:

- OpenAI: jackson -> org.apache.flink.model.openai.shaded.jackson;
httpcomponents -> shaded.httpcomponents
- Triton: jackson -> org.apache.flink.model.triton.shaded.jackson; squareup -> shaded.squareup

11.3 扩展方式

- 新增 Model Provider: 实现 ModelProviderFactory + AsyncPredictFunction，注册 SPI 服务文件
- TritonModelProviderFactory 预留扩展: 未来可根据模型类型路由到不同 ModelFunction
- AbstractTritonModelFunction 未来路线图: 支持 gRPC 协议、批量推理、多输入多输出

十二、关键设计模式与架构思想

工厂方法模式 (Factory Method)

OpenAIModelProviderFactory 和 TritonModelProviderFactory 实现 ModelProviderFactory 接口，通过 SPI 自动发现。工厂根据配置参数创建具体的 AsyncPredictFunction 实例，解耦实例化逻辑。OpenAI 工厂还包含 URL 路由逻辑，自动选择 Chat 或 Embedding 实现。

模板方法模式 (Template Method)

AbstractOpenAIModelFunction.asyncPredict() 定义骨架流程 (null检查 -> token控制 -> 调用子类)，子类 OpenAIChatModelFunction 和 OpenAIEmbeddingModelFunction 实现 asyncPredictInternal() 具体逻辑。AbstractTritonModelFunction 同理。

策略模式 (Strategy)

ErrorHandlingStrategy 枚举 (RETRY/FAILOVER/IGNORE)
允许用户配置不同的错误处理策略。ContextOverflowAction 枚举 (6种)
控制上下文溢出时的处理行为。两者均通过配置项动态选择。

异步编程模式 (Async)

全部基于 AsyncPredictFunction，返回 CompletableFuture。OpenAI 使用官方 SDK 的异步 API；Triton 使用 OkHttp 的 enqueue() 异步回调。实现非阻塞的模型推理调用。

对象池模式 (Object Pool)

OpenAIUtils 和 TritonUtils 实现引用计数的 HTTP 客户端池。相同配置的多个 Subtask 共享同一客户端实例，避免高并行度下创建过多连接。synchronized 保证线程安全。

SPI 服务发现

两个工厂通过 META-INF/services 注册为 Flink Table Factory，实现插件化模型提供者。用户通过 CREATE MODEL 语句指定 provider，Flink 自动通过 SPI 发现对应工厂。

适配器模式 (Adapter)

TritonTypeMapper 作为类型适配器，将 Flink LogicalType 转换为 Triton DataType，并实现双向序列化/反序列化。弥合了两个系统之间的类型差异。

分层异常体系

Triton 模块设计了 TritonException 基类和 4 个子类的异常层次，每个异常携带 isRetryable() 和 ErrorCategory 信息。使上层可以根据异常类型做精确的重试和降级决策。

